Setup

Here is a comprehensive and structured documentation for the first half of your gMSA attack write-up (Setup through BloodHound Enumeration). I have mapped out exactly where your provided screenshots should go so the narrative flows logically and matches your lab environment.

Credential Dumping: The ReadGMSAPassword Attack

Understanding Group Managed Service Accounts (gMSA)

A Group Managed Service Account (gMSA) is a specialized Active Directory (AD) account designed to run automated services securely. Introduced in Windows Server 2012, gMSAs eliminate the need for administrators to manually manage and update service account passwords. Instead, Active Directory handles the password generation and rotates it automatically (defaulting to every 30 days).

While gMSAs drastically improve security by preventing weak passwords and mitigating standard Kerberoasting attacks, they introduce a new attack vector: the **ReadGMSAPassword** permission. If access controls are misconfigured, an attacker who compromises an account with this permission can extract the gMSA's plaintext password or NT hash. This allows the attacker to impersonate the highly privileged service account, leading to lateral movement or domain escalation.

Lab Setup & Prerequisites

To successfully demonstrate this attack, we first need to configure a vulnerable gMSA environment in our Active Directory lab (`lab.local`).

Step 1: Create the User and Security Group

First, we create a standard user and a dedicated security group. Any user or computer added to this group will be granted the permission to retrieve the gMSA password.

In this lab, we are creating a user named `bheem` and adding both the user and the target computer (`DESKTOP-ULCTEDH$`) to the `gmsa_group`.

```
DC01 [Running] - Oracle VirtualBox
Administrator: C:\Windows\system32\cmd.exe - powershell

C:\Users\naruto_uzumaki07>net user bheem Password@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\naruto_uzumaki07> New-ADGroup -Name "gmsa_group" -GroupScope Global -GroupCategory Security
PS C:\Users\naruto_uzumaki07> Add-ADGroupMember -Identity "gmsa_group" -Members "bheem"
PS C:\Users\naruto_uzumaki07> Add-ADGroupMember -Identity "gmsa_group" -Members "DESKTOP-ULCTEDH$"
```

To ensure the group was populated correctly, we can verify its members.

```
PS C:\Users\naruto_uzumaki07> New-ADGroup -Name "gmsa_group" -GroupScope Global -GroupCategory Security
PS C:\Users\naruto_uzumaki07> Add-ADGroupMember -Identity "gmsa_group" -Members "bheem"
PS C:\Users\naruto_uzumaki07> Add-ADGroupMember -Identity "gmsa_group" -Members "DESKTOP-ULCTEDH$"
PS C:\Users\naruto_uzumaki07> Get-ADGroupMember -Identity "gmsa_group" | Select-Object Name, SamAccountName

Name            SamAccountName
----            --------------
DESKTOP-ULCTEDH DESKTOP-ULCTEDH$
bheem           bheem
```

Step 2: Generate the KDS Root Key

Active Directory uses the Key Distribution Service (KDS) Root Key to securely generate gMSA passwords. Before creating a gMSA, a KDS root key must exist in the domain.

Note: In a production environment, you typically wait 10 hours for the key to replicate. In a lab, you can bypass this wait time by setting the effective time to the past (e.g., -10 hours or -30 days).

Before managing these keys, ensure the Active Directory RSAT tools are installed on your management machine.

```
PS C:\Windows\system32> Add-WindowsCapability -Online -Name Rsat.ActiveDirectory.DS-LDS.Tools~~~~0.0.1.0


Path          :
Online        : True
RestartNeeded : False



PS C:\Windows\system32>
PS C:\Windows\system32> Add-KdsRootKey -EffectiveTime ((Get-Date).AddHours(-10))

Guid
----
ddb3d4b1-df59-8d54-d3e7-2da720cf8521


PS C:\Windows\system32> Get-KdsRootKey


AttributeOfWrongFormat :
KeyValue               : {22, 188, 167, 175...}
EffectiveTime          : 2/22/2026 4:54:53 AM
CreationTime           : 2/22/2026 2:54:54 PM
IsFormatValid          : True
DomainController       : CN=DC01,OU=Domain Controllers,DC=lab,DC=local
ServerConfiguration    : Microsoft.KeyDistributionService.Cmdlets.KdsServerConfiguration
KeyId                  : 34c57616-8aaa-2ecf-303d-4d61dca23116
VersionNumber          : 1

AttributeOfWrongFormat :
KeyValue               : {63, 140, 152, 187...}
EffectiveTime          : 2/22/2026 5:18:28 AM
CreationTime           : 2/22/2026 3:18:28 PM
IsFormatValid          : True
DomainController       : CN=DC01,OU=Domain Controllers,DC=lab,DC=local
ServerConfiguration    : Microsoft.KeyDistributionService.Cmdlets.KdsServerConfiguration
KeyId                  : ddb3d4b1-df59-8d54-d3e7-2da720cf8521
VersionNumber          : 1
```

(Optional Context): If you are setting this up on an isolated lab Domain Controller and want to ensure immediate effectiveness without replication delays, setting the time back 30 days is a common administrative trick.

```
PS C:\Windows\system32>
PS C:\Windows\system32> Add-KdsRootKey -EffectiveTime ((Get-Date).AddDays(-30))

Guid
----
d3047b6c-7276-7a8b-0076-80a220952798


PS C:\Windows\system32> Get-KdsRootKey


AttributeOfWrongFormat :
KeyValue               : {22, 188, 167, 175...}
EffectiveTime          : 2/22/2026 4:54:53 AM
CreationTime           : 2/22/2026 2:54:54 PM
IsFormatValid          : True
DomainController       : CN=DC01,OU=Domain Controllers,DC=lab,DC=local
ServerConfiguration    : Microsoft.KeyDistributionService.Cmdlets.KdsServerConfiguration
KeyId                  : 34c57616-8aaa-2ecf-303d-4d61dca23116
VersionNumber          : 1

AttributeOfWrongFormat :
KeyValue               : {63, 140, 152, 187...}
EffectiveTime          : 2/22/2026 5:18:28 AM
CreationTime           : 2/22/2026 3:18:28 PM
IsFormatValid          : True
DomainController       : CN=DC01,OU=Domain Controllers,DC=lab,DC=local
ServerConfiguration    : Microsoft.KeyDistributionService.Cmdlets.KdsServerConfiguration
KeyId                  : ddb3d4b1-df59-8d54-d3e7-2da720cf8521
VersionNumber          : 1

AttributeOfWrongFormat :
KeyValue               : {173, 134, 147, 216...}
EffectiveTime          : 1/23/2026 3:49:58 PM
CreationTime           : 2/22/2026 3:49:58 PM
IsFormatValid          : True
DomainController       : CN=DC01,OU=Domain Controllers,DC=lab,DC=local
ServerConfiguration    : Microsoft.KeyDistributionService.Cmdlets.KdsServerConfiguration
KeyId                  : d3047b6c-7276-7a8b-0076-80a220952798
VersionNumber          : 1
```

Step 3: Create the gMSA Account

With the Key Distribution Service active, we can now create the actual Group Managed Service Account. We will name this account `MyGMSA`.

Crucially, we bind the `PrincipalsAllowedToRetrieveManagedPassword` attribute to our `gmsa_group`. This is the exact configuration that creates our attack vector!

```
PS C:\Users\naruto_uzumaki07> Restart-Service KdsSvc -Force
PS C:\Users\naruto_uzumaki07> Restart-Service ADWS -Force
PS C:\Users\naruto_uzumaki07> New-ADServiceAccount -Name "MyGMSA" -DNSHostName "DC0
1.LAB.LOCAL" -PrincipalsAllowedToRetrieveManagedPassword "gmsa_group" -Server local
host
PS C:\Users\naruto_uzumaki07>
```

We can verify the account creation and inspect its properties to confirm that our security group has the rights to fetch the password.

```
PS C:\Users\naruto_uzumaki07> New-ADServiceAccount -Name "MyGMSA" -DNSHostName "DC01.LAB.LOCAL" -PrincipalsAllowedToRetrieveManagedPassword "gmsa_group" -Server localhost
PS C:\Users\naruto_uzumaki07> Get-ADServiceAccount MyGMSA -Properties PrincipalsAllowedToRetrieveManagedPassword


DistinguishedName                          : CN=MyGMSA,CN=Managed Service
                                             Accounts,DC=lab,DC=local
Enabled                                    : True
Name                                       : MyGMSA
ObjectClass                                : msDS-GroupManagedServiceAccount
ObjectGUID                                 : 76f015a8-e967-48bf-9bd7-eda7956286e9
PrincipalsAllowedToRetrieveManagedPassword : {CN=gmsa_group,CN=Users,DC=lab,DC=local}
SamAccountName                             : MyGMSA$
SID                                        : S-1-5-21-2216088209-294889007-2916038
                                             325-1181
UserPrincipalName                          :



PS C:\Users\naruto_uzumaki07>
```

Step 4: Configure Remote Access and SPNs

To make this service account useful (and exploitable via lateral movement), we need to grant it remote management privileges and assign a Service Principal Name (SPN) for Kerberos authentication.

We will add `MyGMSA$` to the **Remote Management Users** group and register an SPN for a custom service on our target desktop.

```
PS C:\Users\naruto_uzumaki07> Add-ADGroupMember -Identity "Remote Management Users"
 -Members "MyGMSA$"
PS C:\Users\naruto_uzumaki07> setspn -a customerservice/DESKTOP-ULCTEDH.lab.local l
ab.local\MyGMSA
Checking domain DC=lab,DC=local

Registering ServicePrincipalNames for CN=MyGMSA,CN=Managed Service Accounts,DC=lab,
DC=local
        customerservice/DESKTOP-ULCTEDH.lab.local
Updated object
PS C:\Users\naruto_uzumaki07>
```

Step 5: Install the gMSA on the Target Computer

Finally, the target computer must locally install the service account so it can officially retrieve the password from the Domain Controller. We do this using the `Install-ADServiceAccount` cmdlet and verify it with `Test-ADServiceAccount`.

```
PS C:\Windows\system32> Install-ADServiceAccount -Identity "MyGMSA"
PS C:\Windows\system32> Test-ADServiceAccount -Identity "MyGMSA"
True
PS C:\Windows\system32>
```

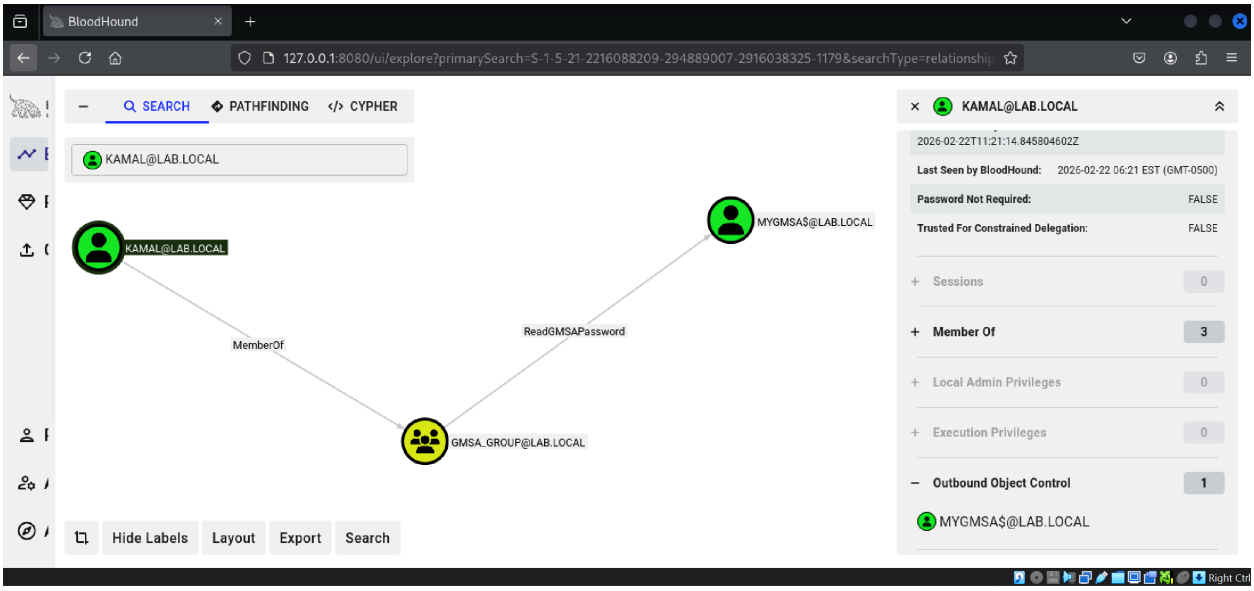
Enumeration Phase

BloodHound – Hunting for Weak Permissions

With our misconfigured environment set up, we step into the shoes of the attacker. Assuming an attacker has compromised a standard domain user (in this case, `kamal@lab.local` or `bheem@lab.local`), they will run standard Active Directory enumeration tools like BloodHound to map out the network.

Note on your lab data: Your setup images show the user `bheem` being added to the group, but your BloodHound graph shows the user `KAMAL` navigating the attack path. If Kamal is also a member of that group in your lab, this is perfectly fine!

By running a SharpHound ingestor and loading the data into BloodHound, the attacker uncovers a direct path to privilege escalation. The graph clearly shows that the compromised user is a member of `GMSA_GROUP@LAB.LOCAL`, which holds the **ReadGMSAPassword** privilege over `MYGMSA$@LAB.LOCAL`.



This visual confirmation is the green light for the attacker to proceed with credential extraction.